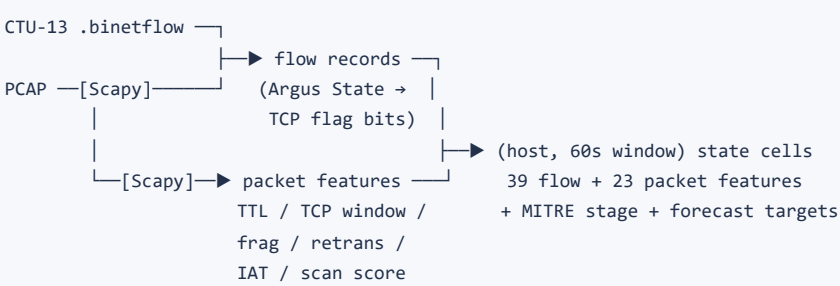
Architecture — AI Network Attack Forecasting

PS 26153 · NTRO · World-model based predictive cyber defence · Team: Git with It

1. Problem framing

Flow classifiers answer "is this flow malicious". The PS asks a different question: *given how this network has been behaving, where is it heading?* That requires a model of state transitions, `P(S_t+1 | S_t)`, that can be run forward without observations. Everything below follows from that requirement.

2. Pipeline

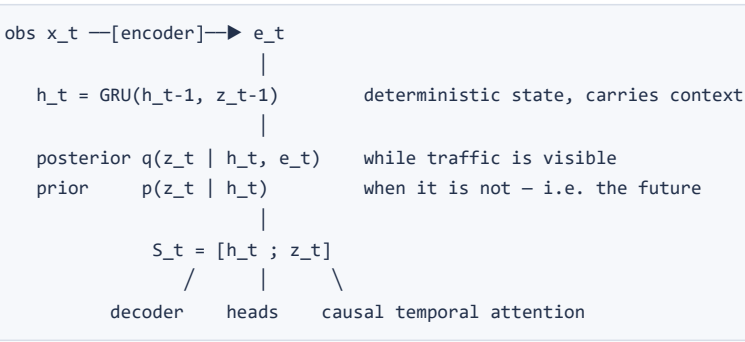


State definition. One state `S_t` is everything a single internal host did in one 60-second window. Host-level rather than network-level because "will the network be attacked" is not actionable, while "is *this* host on a trajectory to exfiltration" is — and it multiplies training sequences by the host count.

Two feature levels, joined on an absolute window grid (`epoch // width`). A PCAP and a flow export of the same capture begin at different timestamps, so a per-file relative grid silently misaligns them.

Stage labels come from CTU-13's own flow annotations (`-CC16-`, `-SPAM`, `-Binary-Download`, `-Attempt`, `-DNS`) mapped onto ATT&CK's five stages by a single inspectable table; documented behavioural rules fill the gaps, which in practice means lateral movement. A cell takes the *dominant* stage among its malicious flows, not the furthest-along one — otherwise a spam bot's first spam flow pins every later window to Exfiltration and progression disappears.

3. World model



Recurrent state-space model, ~138k parameters, CPU-only.

Why this is a world model, testably. Training fits the prior to the posterior via a KL term with a free-nats floor. Once they agree, the prior alone is a simulator: sample `z`, feed it back through the GRU, and the model produces network states it never observed. The prediction heads sit on the latent state, not the observation, so they apply unchanged to imagined states. `tests/smoke_model.py` asserts the property directly — an imagination-only loss must produce **zero gradient in the encoder**. If the rollout were peeking at observations, that test fails.

Objective. Reconstruction (forces the latent to encode traffic) + KL (fits the prior, which is what makes imagination valid) + supervised infiltration and stage heads + an **imagination loss** that supervises the heads on rolled-out states, so multi-step forecasting is trained rather than hoped for.

Forecast. K independent rollouts from the current latent; the mean is the forecast and the 10th–90th percentile spread is the uncertainty band that widens as evidence runs out.

4. Explainability

Three channels, because one is easy to fool yourself with:

Channel	Answers	Method
Feature attribution	which measurements drove the score	integrated gradients from an "average host" baseline; plain gradient × input collapsed to ~1e-3 on confident predictions, which is exactly when an explanation is needed
Temporal attention	which earlier windows mattered	the causal attention weights the heads actually consumed, masked so step i cannot see $i+1$
Predicted state delta	what the model expects to change	decoder output on the imagined state, in real units

The dashboard also splits attribution mass between flow-level and packet-level features, showing when the packet level is carrying the decision.

5. Evaluation design

Two different questions, reported separately:

- **Temporal split** (primary) — train on the first 70% of every capture, forecast its future, with a guard band so no sequence straddles a boundary. This is the deployment shape: a system installed on a network learns that network.
- **Family holdout** — train on Neris/Rbot/Sogou, test on Virut/Murlo. Asks whether the model transfers to unseen malware.

Baselines get identical features, scaler and labels. The stacked logistic regression additionally sees the same 8-window history the world model does, so any gap is attributable to learned dynamics rather than a larger input. Stage forecasting is compared against **persistence** ("assume nothing changes") in two variants: one given the ground-truth current stage (an oracle, not deployable) and one repeating the model's own inferred stage (like-for-like).

6. Two findings that shaped the build

CTU-13 has no pre-infection baseline. Across all seven prepared scenarios, infected hosts are malicious in *every* window they appear — 856 malicious cells, zero benign windows belonging to a compromised machine. The captures were made by running live malware, so "will this host be attacked" is close to degenerate: 101 of 11,388 training sequences contain any positive, and 98 of those are positive at every step. What *is* forecastable is kill-chain progression — 138 real stage transitions between consecutive windows. Stage forecasting is therefore the headline task, with binary infiltration reported as detection transfer.

A PCAP can leak the label. CTU-13's conveniently sized capture for scenario 1 contains only the infected host's traffic, and only part of the timeline. Joined onto the state cells, `has_packet_features` became a perfect label proxy — the model scored 0.9995 on training windows and 0.003 on every later window of the same host, having learned the metadata rather than the behaviour. Packet features are now opt-in and only enabled when packet coverage is label-independent (a full-traffic capture); the extractor still always runs for uploaded PCAPs. Removing the shortcut forced the model onto real behavioural features and detection F1 rose from 0.23 to 0.98 — the leak was making the model *worse*, not better.

A third, smaller one worth mentioning: the world model's prior-error "surprise" signal is *anti*-correlated with compromise here (ROC-AUC 0.375; mean 0.35 on malicious windows against 0.49 on benign). Botnet traffic is more predictable than human traffic. Read in the right direction that is useful — low surprise on a flagged host corroborates automation — so it ships as a labelled novelty channel rather than being folded into the risk score.

7. Decision support surface

FastAPI + a single self-contained HTML page: no CDN, no external font, no telemetry, charts drawn on canvas. Runs with the network cable unplugged.

View	What it answers	How it is computed
Triage list	which host first	every host scored in one batched pass — 149 hosts in 0.5 s
Forecast + replay	is this host heading somewhere bad	`observe` once for all latent states, then one batched `imagine` from every timestep — 145 replay frames in ~2 s
Kill-chain strip	what has this host been doing all capture	one block per window, coloured by inferred stage, dataset labels ticked underneath
Topology	where does it sit in the network	separate `(src, dst)` edge table; node colour is the same risk score
Explanations	why	integrated gradients, causal attention, decoded state delta
Upload	does it work on my traffic	PCAP → flows reconstructed → both feature levels → same forecast

Two engineering notes worth defending:

Replay is batched, not looped. A frame per minute could have been a request per minute; instead `observe` yields the latent state at every timestep in one pass and all rollouts run as a single batch. The difference is 2 seconds against several minutes, and it is what makes playback usable in front of an audience.

The topology view is deliberately truncated. The bot in scenario 1 contacted 1,850 external peers; drawing them all produces an unreadable hairball and an $O(n^2)$ layout that stalls the browser. Nodes are selected by priority — flagged hosts, then the busiest malicious peers, then volume — and the untruncated counts are reported as figures instead, which state the fan-out more clearly than the picture could.